Ardent Pulse — Quick Start Guide

Ardent Pulse — Quick Start Guide

Anomaly detection on your microcontroller in under 30 minutes.

Page 1 — What is Ardent Pulse?

Ardent Pulse is a C99 SDK for real-time anomaly detection on microcontrollers.

Core properties: - **24 bytes RAM** per Z-Score detector - **< 1 µs latency** per sample at 80 MHz - **Zero malloc** — stack allocation only - **Zero external dependencies** — pure C99 - **Testable on PC** before touching any hardware

Supported targets: ESP32, STM32, RISC-V, any MCU via HAL abstraction.

The 3 detectors:

Detector	Catches	RAM
Z-Score (Welford 1962)	Sudden spikes, shocks, voltage surges	24 bytes
EWMA Drift	Slow drift, aging sensors, bearing wear	24 bytes
MAD	Vibration, ECG, noisy industrial signals	~200 bytes

> **Run Z-Score + EWMA Drift in parallel.** They are complementary: Z-Score catches sudden events, EWMA Drift catches slow trends. Z-Score is blind to slow drift because its running mean follows it.

Page 2 — First detection on PC (5 minutes)

No hardware required.

Requirements: GCC + Make (Linux/Mac native — Windows: MSYS2 or WSL)

```
# Step 1 — Compile and run all tests
cd edge-core/tests
make
# Expected: 391/391 tests passed (13 suites)
```

Understand the test output:

```
[PASS] zscore: spike detection at +5σ
[PASS] zscore: no false positive in warm-up
[PASS] drift: slow ramp +0.05/sample detected
[PASS] mad: robust to outliers
...
391/391 tests passed
```

Your first detector in a C file:

```c
#include "ardent/zscore.h"
#include "ardent/drift.h"

int main(void) {
    ArdentZScore spike;
    ArdentDrift  drift;

    ard_zscore_init(&spike, 3.0f, 30);        // 3σ, 30-sample warm-up
    ard_drift_init(&drift, 0.1f, 0.01f, 0.5f); // fast α, slow α, threshold

    for (int i = 0; i < 100; i++) {
        float sample = read_sensor(i);
        bool s = ard_zscore_update(&spike, sample);
        bool d = ard_drift_update(&drift, sample);
        if (s) printf("SPIKE at sample %d\n", i);
        if (d) printf("DRIFT at sample %d\n", i);
    }
    return 0;
}
```

Compile: `gcc main.c edge-core/src/zscore.c edge-core/src/drift.c -Iedge-core/include -DARD_NATIVE_TEST -o demo && ./demo`

Page 3 — Flash to ESP32 (20 minutes)

Requirements: PlatformIO CLI (`pip install platformio`)

Wiring (MPU-6050 accelerometer):

```
MPU-6050    →    ESP32
VCC         →    3.3V
GND         →    GND
SDA         →    GPIO13
SCL         →    GPIO14
```

```
AD0         →    GND   (I2C address = 0x68)
```

Configuration:

```
cd edge-core/examples/esp32/zscore_demo
cp src/config.h.example src/config.h
```

Edit `src/config.h`:

```
#define WIFI_SSID     "your-wifi-ssid"
#define WIFI_PASSWORD "your-wifi-password"
#define MQTT_HOST     "192.168.1.x"   // your Mosquitto broker IP
#define MQTT_USER     "ardent-device"
#define MQTT_PASS     "your-password"
```

Flash and monitor:

```
# IMPORTANT: use board=esp32dev (not esp32cam — PSRAM crash)
pio run --target upload
pio device monitor --baud 115200
```

Expected serial output:

```
[ARDENT] Pulse initialized
[ARDENT] WiFi connected
[ARDENT] MQTT connected
[ARDENT] accel: 9.81 (NORMAL)
[ARDENT] ANOMALY DETECTED (z=4.23) — publishing to MQTT
```

Page 4 — Integrate in your own firmware

Minimal integration (copy-paste ready):

```
#include "ardent/zscore.h"

// Declare at file scope (static = no heap)
static ArdentZScore detector;

void setup(void) {
    // threshold_sigma=3.0, warmup=30 samples
    ard_zscore_init(&detector, 3.0f, 30);
}

void loop(void) {
    float value = read_adc_channel(0);          // your sensor
    bool anomaly = ard_zscore_update(&detector, value);

    if (anomaly) {
        float z = ard_zscore_get_zscore(&detector, value);
        float mean = ard_zscore_get_mean(&detector);
```

```
        float stddev = ard_zscore_get_stddev(&detector);
        // handle anomaly: GPIO, MQTT, LED, buzzer...
    }
}
```

Parameter tuning:

Parameter	Low value	High value	Start with
`threshold_sigma`	More sensitive (more false positives)	Less sensitive (misses small anomalies)	`3.0f`
`warmup_samples`	Faster start, less stable baseline	Slower start, stable baseline	`30`
`window_size`	Adapts faster to shifts	More memory of history	`0` (infinite)

To calibrate automatically on your real data: use Ardent Forge (included in Full Stack package) — `uv run forge run --config configs/my_sensor.yaml`
